

RF-Based UAV Signal Classification with Random Forest and Convoluted Neural Networks

Ilpo Aunio

October 2025

1 Introduction

The war in Ukraine has demonstrated the increasing use of unmanned aerial vehicles (UAVs) in modern warfare. Counter-UAV (C-UAV) technologies designed to detect and counter these threats, are still in development due to the complex electromagnetic environment of the battlefield. In particular, detecting UAVs via their RF emissions is challenging because of signal clutter from telecommunications, radar systems, and electronic warfare jammers.

Machine learning (ML) approaches are suited for this task, as they can learn complex signal patterns and maintain performance even in low signal-to-noise (SNR) conditions. In this report, we compare the performance of Random Forests and Convolutional Neural Networks (CNNs) for UAV detection across varying SNR levels.

2 Problem Formulation

The detection of UAV RF signals can be formulated as a supervised multi-class classification problem. The objective is to classify RF-signal segments either into the correct drone class or as noise. The model is required to work in real time utilizing hardware limited to edge computing.

2.1 Dataset

The dataset used in this project was obtained from Kaggle [1], where RF signals were recorded in an anechoic chamber using a LogPer antenna and captured with an Ettus Research USRP B210[2]. The signals were initially sampled at 56 MHz and downsampled to 14 MHz using SciPy. The lack of real-world samples means results may not generalize outside lab conditions. A single datapoint in the used dataset consists of a continuous signal segments of length 16384, along with their spectrogram, SNR, and duty cycle[1]. Table 1 shows the statistical representation of the classes in the dataset with the class name being the transmitter.

Table 1: Number of datapoints per class in the dataset

Class ID	Class Name	Number of Samples
0	DJI	2,194
1	FutabaT14	6,938
2	FutabaT7	3,661
3	Graupner	6,481
4	Noise	52,552
5	Taranis	16,546
6	Turnigy	10,333

The table shows a imbalance between datapoints being overrepresented by noise which has to be managed in the training data split to avoid overfitting on the noise class.

3 Methods

In this project, we selected the Random Forest as a benchmark and a Convolutional Neural Network (CNN) to investigate the potential benefits of learning time-frequency patterns directly from the data. To ensure a fair comparison, the hyperparameters of the Random Forest were tuned using random search.

3.1 Random forest

The model was chosen in this project because of its robustness against noise and ability to handle nonlinearities which are present in the samples. In addition the model is computationally efficient and feature based which makes it a good benchmark against the CNN.

3.1.1 Feature extraction

For classifying drones, a key distinguishing factor is their unique frequency signatures. To leverage this, we transform the raw signal into the frequency domain using the Fast Fourier Transform (FFT) and extract features suitable for machine learning. The process is as follows:

1. **Signal Preparation:** Combine the in-phase (I) and quadrature (Q) components into a single complex-valued signal.
2. **Frequency Transformation:** Compute the FFT of the complex signal to obtain both magnitude and phase information.
3. **Feature Construction:** Downsample the magnitude and phase components and compute summary statistics, including mean, standard deviation, maximum, median, and average.

The resulting feature vector captures the essential frequency-domain characteristics of the signal and is passed to the random forrest trainer.

3.1.2 Training

Model training data was split 80 percent for model training and 20 percent for evaluation. Splitting was done randomly with balanced class to counteract overfitting for the noise class. The Random Forrest function used was the scikit-learn RandomForestClassifier.

Hyperparameter tuning was conducted by utilizing `RandomizedSearchCV` in combination with scikit-learn's `StratifiedKFold`. Due to limited computational resources, only a coarse search was performed. The search space for the hyperparameters is summarized in Table 2.

Table 2: Hyperparameter search space for random search cross-validation.

Hyperparameter	Search values
<code>n_estimators</code>	100, 200, 500
<code>max_depth</code>	10, 20, None
<code>max_features</code>	sqrt, log2, None
<code>class_weight</code>	balanced, None

3.2 CNN

Convolutud neural networks being able to learn features on its own makes it possible to use the drone data without major processing and thus the model being more flexible in real world applications. To get a valid

feature set for the neural network we will use the datasets provided spectrogram images which has the In-Phase and the Quadrature signals on a different image. These are then passed to the neural network as a two channel input.

3.3 CNN Model Construct and Evaluation

The network architecture, implemented using PyTorch, is composed of several convolutional blocks. Each block contains a convolutional layer to extract local time-frequency patterns with a kernel size of 3, followed by batch normalization to stabilize training and allow higher learning rates. MaxPooling layers are used to reduce spatial dimensions reducing computation cost, provide translational invariance which means the network can see patterns that appear in a slightly different location. Maxpooling also increases the network’s receptive field which means the last convolutional layer sees an 8x8 area of the original image. As the network deepens, the number of feature maps increases, enabling the model to capture more complex patterns in the RF signals. Our model uses three convolutional layers. The first convolutional layer extracts signal edges and simple frequency patterns. Second layer then identifies harmonics and modulations. The final convocational layer finds combinations of patterns for specific drones. Using this three layered approach strikes a balance in accuracy and computational efficiency required for edge computing restrictions.

The output of the convolutional layers is then fed into fully connected layers, which combine the learned spatial features and produce the final class predictions. To prevent overfitting, dropout is applied in the fully connected layers which turns of 50% of neurons during training passes.

For training, the CrossEntropyLoss function is used, which is suitable for multi-class classification tasks where each input belongs to a single class. This loss function penalizes the model when it assigns low probability to the correct class. The Adam optimizer is employed to update network parameters. Adam adapts learning rates individually for each parameter which allows faster convergence on high-dimensional and noisy RF data compared to standard stochastic gradient descent.

The training data was split with a traditional 80-10-10 split with 80 percent of samples being in training and 10 percent in evaluation and test sets. This split provides a good balance in the amount of samples being allocated to training and having enough left over for a accurate evaluation and test set.

During training, the network iterates over multiple epochs. In each iteration, input spectrograms are passed through the network (forward pass) to generate predicted class logits. The loss is then computed, and gradients are backpropagated (backward pass) to update the model parameters. Training accuracy is computed by counting correct predictions, while performance on a separate validation set is monitored using `torch.no_grad()` to prevent gradient calculations. Input tensors are dynamically reshaped to include a channel dimension if needed, ensuring compatibility with the convolutional layers. Both training and validation loss and accuracy are recorded for each epoch, and the model can leverage GPU acceleration via CUDA for faster training.

4 Results

4.1 Random Forest Results

Hyperparameter tuning of the Random Forest model resulted in a 10% overall accuracy improvement. The best parameters obtained from the random search were:

```
n_estimators = 500, max_features = None, max_depth = 20, class_weight = None.
```

After interpreting the results, the `class_weight` parameter was adjusted to `balanced` in order to mitigate overfitting toward the noise class.

The trained Random Forest model achieved an overall accuracy of 0.82. However, the recall of 0.60 3 is considered insufficient for real-world Counter-UAV (C-UAV) applications, where false negatives pose significant safety risks.

4.2 CNN Results

The CNN model was trained over multiple epochs, with experimentation showing that 10 epochs provided a good balance between learning performance and overfitting. The CNN achieved an accuracy of 0.93 and a recall of 0.82 ³. While this recall is still below optimal levels, further improvements could be achieved through additional tuning of the network architecture and enhanced data preprocessing. The confusion marixes of both models figure 1 and figure 2 shows the clear benefit at using CNN over random forrest.

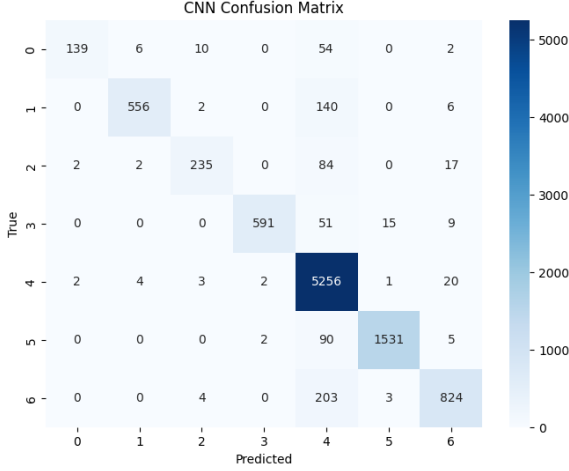


Figure 1: CNN Confusion Matrix

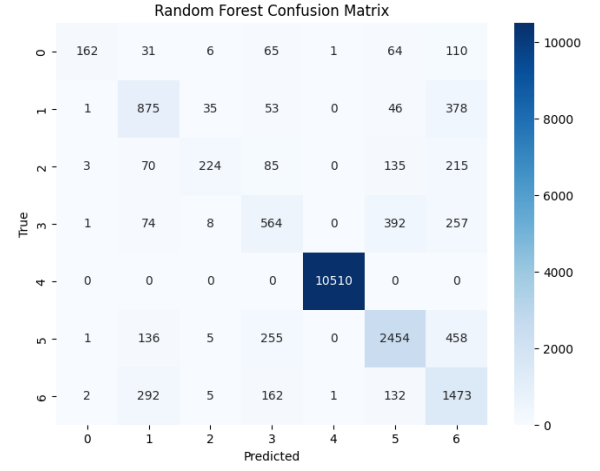


Figure 2: Random Forest Confusion Matrix

Table 3: Comparison of classification performance between Random Forest and CNN models.

Metric	Random Forest	CNN
Accuracy	0.82	0.93
Macro Precision	0.73	0.95
Macro Recall	0.60	0.82
Macro F1-score	0.63	0.88
Weighted Precision	0.84	0.93
Weighted Recall	0.82	0.93
Weighted F1-score	0.82	0.92

5 Conclusion

This study compared the performance of Random Forest and Convolutional Neural Network (CNN) models for RF-based UAV signal classification. The Random Forest model served as a solid baseline due to its simplicity, interpretability, and computational efficiency. However, despite hyperparameter tuning and class balancing, its recall of 0.60 revealed limitations in detecting true UAV signals, which is critical for real-world Counter-UAV (C-UAV) systems.

In contrast, the CNN demonstrated a significant improvement, achieving an accuracy of 0.93 and a recall of 0.82. By learning hierarchical time–frequency patterns directly from spectrograms, the CNN outperformed the feature-engineered Random Forest approach, highlighting the advantages of deep learning in complex RF environments.

While the CNN shows strong potential for operational deployment, further improvements could be realized through additional data augmentation, larger training datasets, and optimized architectures to enhance recall and robustness in low signal-to-noise conditions. Overall, the results indicate that convolutional neural networks provide a more scalable and accurate foundation for future C-UAV RF detection systems.

References

- [1] Stefan Glüge. Noisy drone rf signal classification. <https://www.kaggle.com/datasets/sgluege/noisy-drone-rf-signal-classification/data>, 2023. Accessed: 2025-09-07.
- [2] Stefan Glüge., Matthias Nyfeler., Nicola Ramagnano., Claus Horn., and Christof Schüpbach. Robust drone detection and classification from radio frequency signals using convolutional neural networks. In *Proceedings of the 15th International Joint Conference on Computational Intelligence - NCTA*, pages 496–504. INSTICC, SciTePress, 2023.

GPT-5 was used as a tool to enhance clarity and grammar in text.

rforrest

September 19, 2025

```
[ ]: import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch.utils.data import Dataset, Subset
from sklearn.model_selection import train_test_split
import numpy as np
from numpy.fft import fft, fftfreq, fftshift

FS = 14e6

class DroneSignalsDataset(Dataset):
    def __init__(self, x_iq_tensor, x_spec_tensor, y_tensor, snr_tensor,
        ↪duty_cycle_tensor):
        self.x_iq = x_iq_tensor
        self.x_spec = x_spec_tensor
        self.y = y_tensor
        self.snr = snr_tensor
        self.duty_cycle = duty_cycle_tensor

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.x_iq[idx], self.x_spec[idx], self.y[idx], self.snr[idx],
        ↪self.duty_cycle[idx]

dataset_folder = './sig_data/'

dataset_stats = pd.read_csv(dataset_folder + 'class_stats.csv', index_col=0)
class_names = dataset_stats['class'].values

snr_stats = pd.read_csv(dataset_folder + 'SNR_stats.csv', index_col=0)
snr_list = snr_stats['SNR'].values

# load data
dataset_dict = torch.load(dataset_folder + 'dataset.pt')
```

```

x_iq = dataset_dict['x_iq']
x_spec = dataset_dict['x_spec']
y = dataset_dict['y']
snrs = dataset_dict['snr']
duty_cycle = dataset_dict['duty_cycle']

# create pytorch dataset form tensors
dataset = DroneSignalsDataset(x_iq,x_spec,y,snrs,duty_cycle)
del(x_iq, x_spec, y, snrs, duty_cycle, dataset_dict)

data_loader = torch.utils.data.DataLoader(
    dataset=dataset,
    batch_size=64,
    shuffle=True
)

```

feature explorations

```

[ ]: labels_all = dataset.y
classes, counts = torch.unique(labels_all, return_counts=True)
print("Class stats with num of samples")
for c, count in zip(classes, counts):
    print(f"Class {c.item()}: {count.item()} samples")

x_iq, x_spec, labels, snrs, duty_cycle = next(iter(data_loader))

```

```

Class stats with num of samples
Class 0: 2194 samples
Class 1: 6938 samples
Class 2: 3661 samples
Class 3: 6481 samples
Class 4: 52552 samples
Class 5: 16546 samples
Class 6: 10333 samples

```

```

[5]: def plot_drone_data(spectrogram, iq, title, fs=1.0):
    figsize = (10,7)
    layout = [ [ 'spec_re', 'spec_im'],
                ['spec_re', 'spec_im' ],
                ['iq_re', 'iq_re']]
    fig, axs = plt.subplot_mosaic(layout, figsize=figsize)

    fig.suptitle(title, fontsize=16)

```

```

# plot spectrogram Re and Im
spec_re = axs['spec_re'].imshow(spectrogram[0,:,:])
axs['spec_re'].set_title('Re', fontsize=10)
fig.colorbar(spec_re, ax=axs['spec_re'], location='right', shrink=0.5)

spec_im = axs['spec_im'].imshow(spectrogram[1,:,:])
axs['spec_im'].set_title('Im', fontsize=10)
fig.colorbar(spec_im, ax=axs['spec_im'], location='right', shrink=0.5)

# FFT
N = iq.shape[1]
Isig_ft = fft(iq[0,:])
Qsig_ft = fft(iq[1,:])

freqs = np.fft.fftfreq(N, d=1/fs)
pos_idx = freqs >= 0
freqs = freqs[pos_idx]

Isig_ft = np.abs(Isig_ft[pos_idx])
Qsig_ft = np.abs(Qsig_ft[pos_idx])

axs['iq_re'].plot(freqs, np.abs(Isig_ft), label="I")
axs['iq_re'].plot(freqs, np.abs(Qsig_ft), label="Q")
axs['iq_re'].set_xlabel("Frequency [MHz]")
axs['iq_re'].set_ylabel("Magnitude")
axs['iq_re'].legend()
axs['iq_re'].set_title("FFT of IQ signal")

def batch_plot(x_iq_batch, x_spec_batch, labels_batch, snrs_batch,
    ↪duty_cycle_batch, fs=1.0):
    plotted_classes = set()
    classes_in_batch = labels_batch.unique()

    for i in range(labels_batch.size(0)):
        label = labels_batch[i].item()

        if label not in plotted_classes:
            x_iq = x_iq_batch[i].numpy()
            x_spec = x_spec_batch[i].numpy()
            snr = snrs_batch[i].item()
            duty = duty_cycle_batch[i].item()
            title = f"Class {class_names[label]} | SNR: {snr} | Duty cycle:
            ↪{duty}"

```



```

        plot_drone_data(x_spec, x_iq, title, fs=fs)
        plotted_classes.add(label)

    if len(plotted_classes) == len(classes_in_batch):
        break

def plot_fft_per_class(x_iq_batch, labels_batch, fs=1.0):
    classes_in_batch = sorted(labels_batch.unique().tolist())
    n_classes = len(classes_in_batch)

    mosaic_layout = [
        ['c0', 'c1'],
        ['c2', 'c3'],
        ['c4', 'c5'],
        ['c6', 'c7']
    ]

    fig, axs = plt.subplot_mosaic(mosaic_layout, figsize=(14, 12),
    constrained_layout=True)
    fig.suptitle("FFT of IQ Signals per Drone Class", fontsize=16)

    plotted_classes = set()
    N = x_iq_batch.shape[2] # number of samples

    for i in range(labels_batch.size(0)):
        label = labels_batch[i].item()
        if label not in plotted_classes:
            iq = x_iq_batch[i].numpy()
            I_fft = fft(iq[0,:])
            Q_fft = fft(iq[1,:])
            freqs = np.fft.fftfreq(N, d=1/fs)
            pos_idx = freqs >= 0
            freqs = freqs[pos_idx]
            I_fft = np.abs(I_fft[pos_idx])
            Q_fft = np.abs(Q_fft[pos_idx])

            subplot_name = f'c{classes_in_batch.index(label)}'
            ax = axs[subplot_name]
            ax.plot(freqs, I_fft, label="I", color='blue')
            ax.plot(freqs, Q_fft, '--', label="Q", color='red')
            ax.set_title(f"{class_names[label]}")
            ax.set_xlabel("Frequency [MHz]")
            ax.set_ylabel("Magnitude")
            ax.legend()
            ax.grid(True)

            plotted_classes.add(label)

```

```

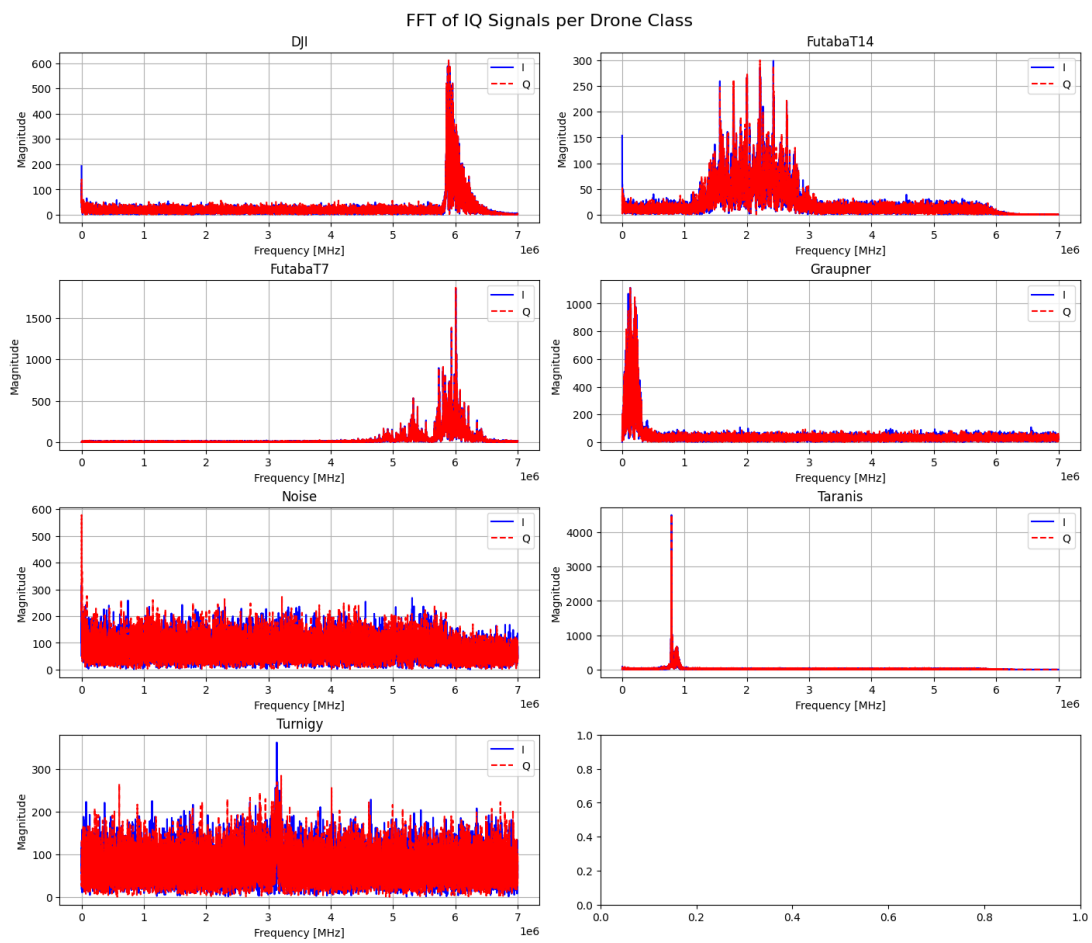
    if len(plotted_classes) == n_classes:
        break

    # Hide unused subplots if fewer than 6 classes
    for idx in range(n_classes, 6):
        axs[f'c{idx}'].axis('off')

plt.show()

plot_fft_per_class(x_iq, labels, fs=14e6 )
#batch_plot(x_iq, x_spec, labels, snrs, duty_cycle, fs=14e6)

```



0.1 FFT Feature Extraction

features for ml are extracted by:

1. Combining I/Q components into a complex signal.

2. Computing the FFT and extracting magnitude and phase.
3. Downsampling magnitude and phase and adding summary statistics (mean, std, max, median, average). The resulting feature vector `X_fft` is ready for machine learning tasks.

```
[ ]: def extract_fft_features(x_iq):
    signal = x_iq[0,:] + 1j*x_iq[1,:]
    spectrum = fft(signal)[:len(signal)//2]
    mag = np.abs(spectrum)
    phase = np.angle(spectrum)

    feats = [
        np.mean(mag),
        np.std(mag),
        np.max(mag),
        np.median(mag),
        np.sum(mag) / len(mag)
    ]

    mag_ds = mag[:128]
    phase_ds = phase[:128]
    return np.concatenate([mag_ds, phase_ds, feats])

X_fft = np.array([extract_fft_features(x) for x in dataset.x_iq.numpy()])
```

```
[7]: snrs_np = dataset.snr.numpy().reshape(-1, 1)
    duty_np = dataset.duty_cycle.numpy().reshape(-1, 1)
    print(X_fft.shape)
    print(snrs_np.shape)
    print(duty_np.shape)

    X = np.hstack([X_fft, snrs_np, duty_np])
    y = dataset.y.numpy()

    print("Feature matrix shape:", X.shape)
```

```
(98705, 133)
(98705, 1)
(98705, 1)
Feature matrix shape: (98705, 135)
```

0.2 Training

radom forrest with automated Hyperparameter search

```
[8]: from sklearn.model_selection import train_test_split

    # X = features, y = labels with 20% test_size
    X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, stratify=y, random_state=42
)

param_dist = {
    "n_estimators": [100, 200, 500],
    "max_depth": [10, 20, None],
    "max_features": ["sqrt", "log2", None],
    "class_weight": ["balanced", None]
}

```

code snippet below finds best hyperparameters via cv randomized search

```

[ ]: from sklearn.model_selection import StratifiedKFold, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42, n_jobs=-1)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
search = RandomizedSearchCV(
    rf, param_distributions=param_dist, n_iter=15,
    cv=cv, scoring="accuracy", verbose=2, n_jobs=-1, random_state=42
)

search.fit(X_train, y_train)
best_rf = search.best_estimator_

print("Best parameters:", search.best_params_)
#Best parameters: {'n_estimators': 500, 'max_features': None, 'max_depth': 20,
↪ 'class_weight': None}

```

Fitting 5 folds for each of 15 candidates, totalling 75 fits

```

[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=100;
total time= 1.5min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=100;
total time= 1.5min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=100;
total time= 1.5min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=100; total time= 1.5min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=100;
total time= 1.5min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=100;
total time= 1.6min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=200;
total time= 2.8min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=200;
total time= 2.8min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=200;

```

```

total time= 2.9min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=200;
total time= 2.9min
[CV] END class_weight=None, max_depth=None, max_features=log2, n_estimators=200;
total time= 3.0min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=100; total time= 1.5min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=100; total time= 1.5min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=100; total time= 1.5min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=100; total time= 1.5min
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
n_estimators=200; total time= 4.9min
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
n_estimators=200; total time= 5.0min
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
n_estimators=200; total time= 5.1min
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
n_estimators=200; total time= 5.2min
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
n_estimators=200; total time= 5.3min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=500; total time= 4.7min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=500; total time= 4.9min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=500; total time= 5.0min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=500; total time= 4.9min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=500; total time= 5.1min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=100; total time= 1.0min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=100; total time= 1.1min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=100; total time= 1.1min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=100; total time= 1.0min
[CV] END class_weight=balanced, max_depth=10, max_features=log2,
n_estimators=100; total time= 1.1min
[CV] END class_weight=None, max_depth=10, max_features=log2, n_estimators=500;
total time= 4.6min
[CV] END class_weight=None, max_depth=10, max_features=log2, n_estimators=500;
total time= 4.6min
[CV] END class_weight=None, max_depth=10, max_features=log2, n_estimators=500;

```

total time= 4.5min
[CV] END class_weight=None, max_depth=10, max_features=log2, n_estimators=500;
total time= 4.6min
[CV] END class_weight=None, max_depth=10, max_features=log2, n_estimators=500;
total time= 4.6min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=200; total time= 3.0min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=200; total time= 2.9min
[CV] END class_weight=None, max_depth=None, max_features=None, n_estimators=200;
total time=37.5min
[CV] END class_weight=None, max_depth=None, max_features=None, n_estimators=200;
total time=36.9min
[CV] END class_weight=None, max_depth=None, max_features=None, n_estimators=200;
total time=37.2min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=200; total time= 2.7min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=200; total time= 2.7min
[CV] END class_weight=None, max_depth=None, max_features=None, n_estimators=200;
total time=37.8min
[CV] END class_weight=None, max_depth=None, max_features=None, n_estimators=200;
total time=37.9min
[CV] END class_weight=balanced, max_depth=20, max_features=log2,
n_estimators=200; total time= 2.8min
[CV] END class_weight=None, max_depth=20, max_features=None, n_estimators=500;
total time=75.4min
[CV] END class_weight=None, max_depth=20, max_features=None, n_estimators=500;
total time=75.7min
[CV] END class_weight=None, max_depth=20, max_features=None, n_estimators=500;
total time=75.0min
[CV] END class_weight=None, max_depth=20, max_features=None, n_estimators=500;
total time=75.9min
[CV] END class_weight=None, max_depth=20, max_features=None, n_estimators=500;
total time=76.6min
[CV] END class_weight=balanced, max_depth=20, max_features=None,
n_estimators=500; total time=76.7min
[CV] END class_weight=balanced, max_depth=20, max_features=None,
n_estimators=500; total time=77.9min
[CV] END class_weight=balanced, max_depth=20, max_features=None,
n_estimators=500; total time=82.9min
[CV] END class_weight=balanced, max_depth=20, max_features=None,
n_estimators=500; total time=83.1min
[CV] END class_weight=balanced, max_depth=20, max_features=None,
n_estimators=500; total time=83.4min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=100; total time=12.0min
[CV] END class_weight=balanced, max_depth=10, max_features=None,

```

n_estimators=100; total time=10.7min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=100; total time=12.0min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=500; total time=50.8min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=500; total time=50.7min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=100; total time=10.7min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=100; total time=11.7min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=500; total time=55.2min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=500; total time=55.3min
[CV] END class_weight=balanced, max_depth=10, max_features=None,
n_estimators=500; total time=55.3min
[CV] END class_weight=None, max_depth=10, max_features=None, n_estimators=200;
total time=16.9min
[CV] END class_weight=None, max_depth=10, max_features=None, n_estimators=200;
total time=15.6min
[CV] END class_weight=None, max_depth=10, max_features=None, n_estimators=200;
total time=14.4min
[CV] END class_weight=None, max_depth=10, max_features=None, n_estimators=200;
total time=14.5min
[CV] END class_weight=None, max_depth=10, max_features=None, n_estimators=200;
total time=12.8min
[CV] END class_weight=balanced, max_depth=None, max_features=None,
n_estimators=500; total time=70.6min
[CV] END class_weight=balanced, max_depth=None, max_features=None,
n_estimators=500; total time=46.7min
[CV] END class_weight=balanced, max_depth=None, max_features=None,
n_estimators=500; total time=47.5min
[CV] END class_weight=balanced, max_depth=None, max_features=None,
n_estimators=500; total time=48.1min
[CV] END class_weight=balanced, max_depth=None, max_features=None,
n_estimators=500; total time=46.3min
Best parameters: {'n_estimators': 500, 'max_features': None, 'max_depth': 20,
'class_weight': None}

```

trains random tree with the best found parameters and saves them with joblib

```

[ ]: from joblib import dump
      from sklearn.ensemble import RandomForestClassifier

      rf = RandomForestClassifier(
          n_estimators=500,
          class_weight='balanced',

```

```

        max_depth=20,
        n_jobs=-1,
        random_state=42
    )
    rf.fit(X_train, y_train)

```

```
[ ]: ['rforrest_model.joblib']
```

0.3 Evaluation

```
[ ]: from sklearn.metrics import accuracy_score, classification_report, \
      ↪confusion_matrix
      from joblib import load

      if not rf:
          rf = load("rforrest_model.joblib")

      y_pred = rf.predict(X_test)

      print("Accuracy:", accuracy_score(y_test, y_pred))

      print(classification_report(y_test, y_pred, target_names=class_names))

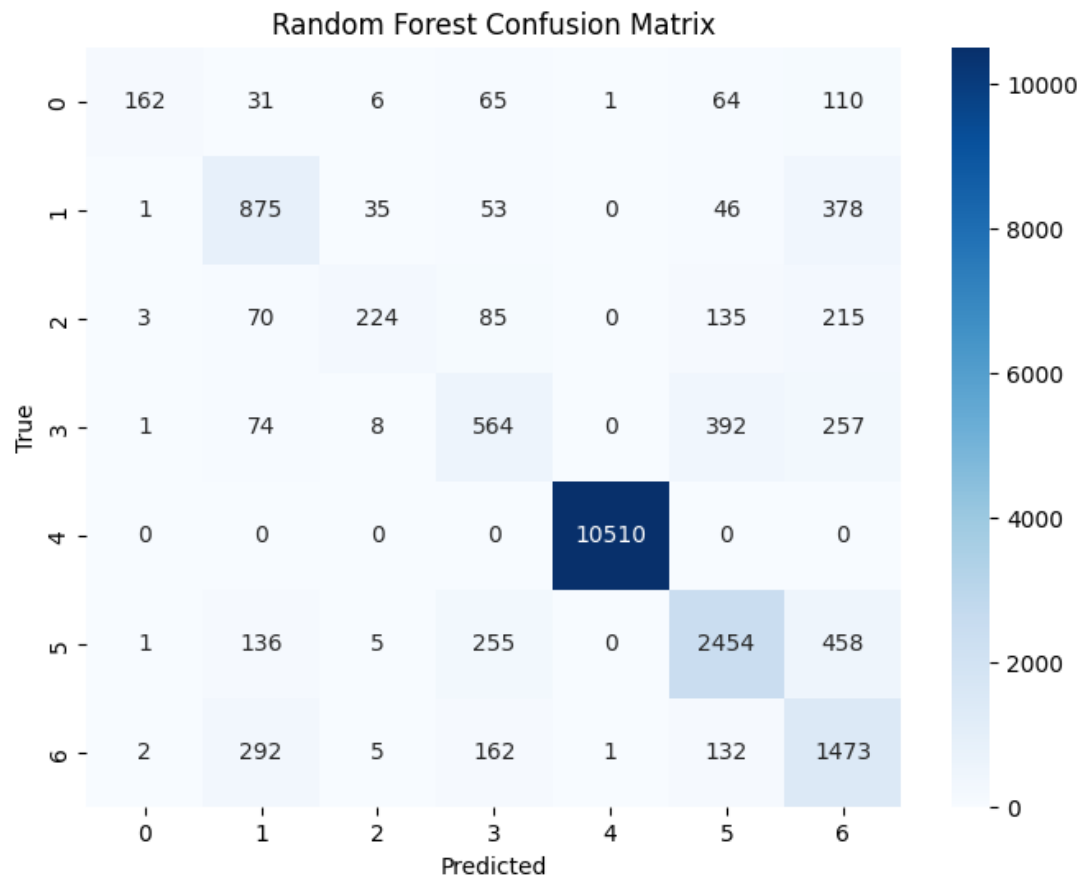
      import matplotlib.pyplot as plt
      import seaborn as sns
      cm = confusion_matrix(y_test, y_pred)
      plt.figure(figsize=(8,6))
      sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
      plt.xlabel("Predicted")
      plt.ylabel("True")
      plt.title("Random Forest Confusion Matrix")
      plt.show()

```

Accuracy: 0.8237677929182918

	precision	recall	f1-score	support
DJI	0.95	0.37	0.53	439
FutabaT14	0.59	0.63	0.61	1388
FutabaT7	0.79	0.31	0.44	732
Graupner	0.48	0.44	0.45	1296
Noise	1.00	1.00	1.00	10510
Taranis	0.76	0.74	0.75	3309
Turnigy	0.51	0.71	0.59	2067
accuracy			0.82	19741
macro avg	0.73	0.60	0.63	19741

weighted avg 0.84 0.82 0.82 19741



10% improvement in accuracy was made thanks to the addition of snr and dutycycle and tuned parameters

cnn

September 26, 2025

0.1 Dataset loading

```
[4]: import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch.utils.data import Dataset, Subset
from sklearn.model_selection import train_test_split
import numpy as np
from numpy.fft import fft, fftfreq, fftshift

FS = 14e6

class DroneSignalsDataset(Dataset):
    def __init__(self, x_iq_tensor, x_spec_tensor, y_tensor, snr_tensor,
    ↪duty_cycle_tensor):
        self.x_iq = x_iq_tensor
        self.x_spec = x_spec_tensor
        self.y = y_tensor
        self.snr = snr_tensor
        self.duty_cycle = duty_cycle_tensor

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.x_iq[idx], self.x_spec[idx], self.y[idx], self.snr[idx],
    ↪self.duty_cycle[idx]

dataset_folder = './sig_data/'

dataset_stats = pd.read_csv(dataset_folder + 'class_stats.csv', index_col=0)
class_names = dataset_stats['class'].values

snr_stats = pd.read_csv(dataset_folder + 'SNR_stats.csv', index_col=0)
snr_list = snr_stats['SNR'].values

# load data
```

```

dataset_dict = torch.load(dataset_folder + 'dataset.pt')

x_iq = dataset_dict['x_iq']
x_spec = dataset_dict['x_spec']
y = dataset_dict['y']
snrs = dataset_dict['snr']
duty_cycle = dataset_dict['duty_cycle']

# create pytorch dataset form tensors
dataset = DroneSignalsDataset(x_iq,x_spec,y,snrs,duty_cycle)
del(x_iq, y, snrs, duty_cycle, dataset_dict)

data_loader = torch.utils.data.DataLoader(
    dataset=dataset,
    batch_size=64,
    shuffle=True
)

```

0.2 CNN Model

```

[ ]: import torch
import torch.nn as nn
import torch.nn.functional as F

class DroneCNN(nn.Module):
    def __init__(self, num_classes):
        super(DroneCNN, self).__init__()

        self.conv1 = nn.Conv2d(2, 16, kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(16)
        self.pool1 = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm2d(32)
        self.pool2 = nn.MaxPool2d(2, 2)

        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.bn3 = nn.BatchNorm2d(64)
        self.pool3 = nn.MaxPool2d(2, 2)

        self.fc1 = nn.Linear(64 * (x_spec.shape[2] // 8) * (x_spec.shape[3] // 8), 256)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(256, num_classes)

    def forward(self, x_spec):

```

```

        x = F.relu(self.bn1(self.conv1(x_spec)))
        x = self.pool1(x)

        x = F.relu(self.bn2(self.conv2(x)))
        x = self.pool2(x)

        x = F.relu(self.bn3(self.conv3(x)))
        x = self.pool3(x)

        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)

    return x

```

0.3 Training Loop

```

[ ]: from torch.utils.data import DataLoader, random_split
total_len = len(dataset)
train_len = int(0.8 * total_len)
val_len   = int(0.1 * total_len)
test_len  = total_len - train_len - val_len

train_dataset, val_dataset, test_dataset = random_split(dataset, [train_len,
    ↪ val_len, test_len])

batch_size = 64

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader   = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader  = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

```

```

[28]: # Model, Loss, Optimizer
num_classes = len(class_names)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Training with: {device}")

model = DroneCNN(num_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def evaluate(model, val_loader, criterion, device):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0

```

```

with torch.no_grad():
    for x_iq, x_spec, y, snr, duty in val_loader:
        x_spec, y = x_spec.to(device), y.to(device)
        if x_spec.dim() == 3:
            x_spec = x_spec.unsqueeze(1)

        outputs = model(x_spec)
        loss = criterion(outputs, y)

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += y.size(0)
        correct += predicted.eq(y).sum().item()

    avg_loss = running_loss / len(val_loader)
    accuracy = 100. * correct / total
    return avg_loss, accuracy

# Training loop
for epoch in range(10):
    model.train()
    running_loss, correct, total = 0.0, 0, 0

    for x_iq, x_spec, y, snr, duty in train_loader:
        x_spec, y = x_spec.to(device), y.to(device)
        if x_spec.dim() == 3:
            x_spec = x_spec.unsqueeze(1)

        optimizer.zero_grad()
        outputs = model(x_spec)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += y.size(0)
        correct += predicted.eq(y).sum().item()

    acc = 100. * correct / total
    train_loss = running_loss / len(train_loader)

    val_loss, val_acc = evaluate(model, val_loader, criterion, device)

```

```

print(f"Epoch {epoch+1}, "
      f"Train Loss: {train_loss:.4f}, Train Acc: {acc:.2f}%, "
      f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")

SAVE_MODEL = True

if SAVE_MODEL:
    torch.save(model.state_dict(), "models/CNN_model_weights.pth")

```

Training with: cuda

```

Epoch 1, Train Loss: 0.9524, Train Acc: 71.70%, Val Loss: 0.6182, Val Acc: 81.88%
Epoch 2, Train Loss: 0.6208, Train Acc: 81.26%, Val Loss: 0.5139, Val Acc: 85.24%
Epoch 3, Train Loss: 0.5120, Train Acc: 84.82%, Val Loss: 0.4688, Val Acc: 87.85%
Epoch 4, Train Loss: 0.4453, Train Acc: 87.03%, Val Loss: 0.3833, Val Acc: 90.25%
Epoch 5, Train Loss: 0.4068, Train Acc: 88.37%, Val Loss: 0.3900, Val Acc: 90.00%
Epoch 6, Train Loss: 0.3750, Train Acc: 89.40%, Val Loss: 0.4083, Val Acc: 90.16%
Epoch 7, Train Loss: 0.3430, Train Acc: 90.21%, Val Loss: 0.3796, Val Acc: 90.83%
Epoch 8, Train Loss: 0.3257, Train Acc: 90.83%, Val Loss: 0.3499, Val Acc: 91.45%
Epoch 9, Train Loss: 0.2984, Train Acc: 91.44%, Val Loss: 0.3376, Val Acc: 91.75%
Epoch 10, Train Loss: 0.2809, Train Acc: 91.76%, Val Loss: 0.3169, Val Acc: 92.12%

```

```

[29]: import torch
from sklearn.metrics import accuracy_score, classification_report, \
      ↪confusion_matrix

model.eval()
y_true = []
y_pred = []

with torch.no_grad():
    for x_iq, x_spec, y, snr, duty in test_loader:
        x_spec = x_spec.to(device)
        y = y.to(device)

        outputs = model(x_spec)

```

```

preds = torch.argmax(outputs, dim=1)

y_true.extend(y.cpu().numpy())
y_pred.extend(preds.cpu().numpy())

print("CNN Accuracy:", accuracy_score(y_true, y_pred))

# Precision, Recall, F1-score
print(classification_report(y_true, y_pred, target_names=class_names))

# Confusion Matrix
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("CNN Confusion Matrix")
plt.show()

```

CNN Accuracy: 0.9251342315874784

	precision	recall	f1-score	support
DJI	0.97	0.66	0.79	211
FutabaT14	0.98	0.79	0.87	704
FutabaT7	0.93	0.69	0.79	340
Graupner	0.99	0.89	0.94	666
Noise	0.89	0.99	0.94	5288
Taranis	0.99	0.94	0.96	1628
Turnigy	0.93	0.80	0.86	1034
accuracy			0.93	9871
macro avg	0.95	0.82	0.88	9871
weighted avg	0.93	0.93	0.92	9871

